

Yellow Phoenix: Semantic Search on an Apple Watch — 86.5% Top-10 Fidelity at 3.9M Documents, Bit-Identical from watchOS to iOS to macOS and Linux

Marc John Sawyer — Independent Researcher · Dragon of Eternal Fire™
marcsawyer76@outlook.com · marcsawyer76@yahoo.com

© 2026 Marc John Sawyer, sole author and copyright holder. Yellow Phoenix™ and Dragon of Eternal Fire™ are trademarks of the author. Engine released under AGPL-3.0.

Version 1.0.2 — 2026-10-03. NOT FOR SUBMISSION. Author-review candidate.

Abstract

We demonstrate semantic document retrieval at million-document scale on hardware no one expects to run it: an Apple Watch. Yellow Phoenix compresses 5.66M academic papers to 512-bit ITQ binary codes (MiniLM embeddings + Iterative Quantization) and searches them with a binary HNSW graph, an exhaustive scan, or graph search re-ranked by a 1024-bit sidecar — all in a memory footprint a wrist computer can hold. On a 3.9M-document prefix, an Apple Watch Series 10 answers with 86.5% top-10 fidelity at 34.8 ms (designated-member protocol); an iPhone answers the same queries at 7.4 ms; and the same engine binary scores bit-identical recall — to the tenth of a point — on watchOS, iOS, macOS, and Linux/x86: one engine, four hosts, two architectures. At 4.9M documents, graph search plus 1024-bit re-ranking reaches 94.0% top-10 at ≈ 0.09 ms on an iPhone — 185 $\times$ faster and $\sim 4\times$ leaner (raw configuration) than our own predecessor edge system (different corpus and queries; see footnote). The measured claim is index fidelity — containment of the reference float index’s own nearest-different-document neighbourhood — and we stress-test it rather than assume it: a three-judge panel (two independent model families plus one same-backbone cross-encoder, shown for completeness) measures 79.6–87.8% fidelity under each judge’s own neighbourhood once structural target absence is conditioned out; the strongest judge’s dissent triggered our pre-registered kill criterion, and we publish the firing, the conditioned numbers, and the re-scoped claim. We report negative results with the same rigor as positive ones: a gate-rescoring model with zero measured effect, a hash-folding scheme that collapses graph recall, and a ground-truth bug we found, fixed, and disclose in a dedicated box.

Keywords: on-device retrieval, binary hashing, HNSW, smartwatch, approximate nearest neighbor search, index fidelity, cross-platform determinism

1. Introduction

The device at the bottom of the wearables ladder — a smartwatch with ~ 1 GB of RAM and no swap — has never been a semantic search platform. Prior wrist-scale demonstrations of our own engine were exact-identity lookups (2.87M fingerprints at ~ 420 ns, 100%), while *semantic* search at 3M documents was refused outright for RAM [19, 20]. This paper closes that gap and, in doing so, makes four contributions:

1. **The device frontier, measured.** Semantic nearest-*different*-document retrieval at 3.9M documents on a wrist (86.5% top-10 fidelity, 34.8 ms), 4.9M on a phone (94.0% with re-ranking, ~ 0.09 ms), with full ef sweeps, latency percentiles, and memory gates — all physical runs, all reproducible from shipped binaries.
2. **A cross-space fidelity protocol with a three-judge validation panel.** Queries and targets are corpus documents; the target is a designated member of the query’s reference top-10 under full-precision embeddings, computed offline; the device searches only binary hashes. Retrieval space $\neq$ grading space. Because the reference neighbourhood is itself a model-dependent object, we submit it to three judges (§4.3): BGE and Qwen3 are the two independent families; the same-backbone cross-encoder ms-marco is shown but excluded as a documented pathological outlier. We scope every recall number as *index fidelity* — the quantity an index can actually promise. The protocol also surfaces two evaluation-mechanics findings that generalize beyond this system: **self-occupancy** (the query’s own record occupies rank 1, making strict single-target top-1 zero by construction) and **structural presence** (recall gaps against absent targets measure the filter, not the index — like-for-like, the index is equally faithful to any reference neighbour that exists).

3. **Portability as a claim.** The same engine binary produces bit-identical recall on four operating systems spanning two ISAs (ARM64 watchOS/iOS/macOS, x86-64 Linux), nine or more independent runs, zero deviation — determinism as engineering evidence, not assumption.
4. **Duplicate-cluster infrastructure (BDC + IPCM + canonical selection).** The measured corpus is 25.6% duplicated; ingest-time cluster maintenance (validated 13,998/13,998 against batch rebuild) and canonical representatives make that structure explicit and usable (§8). Infrastructure, not headline: the formal spec is archived, and the literature sweep (§2) shows incremental duplicate clustering has prior art — our delta is purity-preserving maintenance with audit, which earns its place here but not a standalone paper.

Along the way we report what did not work: a learned gate over hamming-derived features (measured effect: 0.0 pp, with a theoretical ceiling argument for why), fold-of-1024 hashing (excellent for exhaustive scan, fatal for graph traversal), and a ground-truth computation bug (§5.5) that inflated our own earlier exam numbers by up to 55 points at R@1 until found by audit.

THE WALL	MEASURED EVIDENCE	THE INSTRUMENT	STATUS
0 • DATA (own)	the canonical 5.66M-record hash file — the foundation of every number — CLOBBERED mid-study (wrong-space regen, Oct 1)	overnight 2-way sharded rebuild · 5,240-row mid-buffer salvage · 5/5 determinism proofs	REBUILT 12/12 re-encode verified; the incident record is archived (§7)
1 • RAM	semantic tier REFUSED at 3M on a wrist (cascade paper)	memory gate (refusal-as-data) 282 MB graphs · scan-as-right-answer	PASSED 86.5% @ 3.9M on the wrist (34.8 ms)
2 • INFORMATION	512-bit tier plateaus: 96.9% ceiling @ K=200 (99.4% @ K=2000)	a second code, not a bigger one: 1024-bit re-rank sidecar	PASSED 94.0% @ 4.9M, ~0.09 ms (iPhone)
3 • FOLDED HASH	fold-of-1024 collapses graph traversal: 58.9–72.7% vs 82.6%	understood, not fixed; hash objective must match query protocol	MAPPED published as a result (§5.4)
4 • LEARNED GATE	0.0 pp effect in all three training regimes (monotone-feature ceiling)	no independent signal; the 1024 sidecar is the feature that works	MAPPED published as a result (§5.4)
5 • OUR OWN	ground-truth bug, +55 pt inflation, caught by our own audit · kill criterion FIRED	the bug is boxed; the firing is published with the re-scoped claim	OWNED the honesty the paper is built on (§4.3, §5.5)
— • STANDING	the 4.9M graph on a wrist (RAM)	blueprints: flash-resident engine (DiskANN-family ideas, wrist-sized)	STANDING future work (paper 4)

All values from the runs archived with the artifact (benchmark_results/); § references are to this paper. A wall mapped is worth more than a wall hidden.

Figure 1: Six walls, and what crossed them — the paper’s argument in one measured chart. All values from the archived runs; green = passed, purple = owned or rebuilt (our own errors, the data disaster, and the fired kill criterion), amber = mapped and published as results, grey = still standing (future work).

Roadmap. §2 related work; §3 the engine; §4 protocol and evaluation design; §5 results; §6 limitations and threats to validity; §7 verification and artifact.

2. Related Work

Search process. We surveyed (2026-10-03) the ANN/vector-search literature via scholarly search over: on-device and mobile ANN, binary hashing and product quantization, SSD-resident graphs, hardware co-design for ANN, wearable/edge retrieval, and incremental duplicate clustering. Queries combined “approximate nearest neighbor” with “on-device / mobile / smartphone / wearable”, “binary codes / ITQ / product quantization / hamming graph”, “SSD-resident / DiskANN”, and “incremental duplicate clustering / stream deduplication”. We read abstracts and key sections of the closest hits; what follows positions this work against each thread. The negative claim — no published sub-GB on-device *semantic* peer at million-document scale — reflects this process, not an exhaustive proof.

In-memory and SSD-resident ANN. HNSW [Malkov & Yashunin] and its relatives (NSG [Fu et al., VLDB

2019]; HM-ANN, NeurIPS 2020) define the graph-ANN baseline on servers; DiskANN [Subramanya et al., NeurIPS 2019] extends the family to SSD-resident billion-point indexes (Vamana graph, PQ guides, 64 GB RAM class), and FreshDiskANN adds streaming updates. All target workstations or servers with gigabytes to terabytes of memory and watts to spare. Our contribution is not a new ANN algorithm but the *operating point*: the same family at 64 B/record, sub-watt, on a wrist — plus the observation that at this scale the exhaustive scan (a regime the ANN literature treats as a baseline to beat) becomes the right answer for flash-backed devices: zero build cost, 100%-deterministic, memory-light.

Quantization for retrieval. ITQ [Gong & Lazebnik 2011], which we adopt, and PQ/OPQ [Jégou et al.] are the canonical compression families; the learning-to-hash literature [Wang et al., survey] documents their trade-offs. Our two-code architecture (512-bit graph code + 1024-bit re-rank sidecar) is an instance of coarse-propose / fine-verify, distinguished here by being *measured* at wearable scale and by the negative result that a folded code cannot substitute for the graph code (§5.4). Hardware co-design for ANN (e.g. ACRONYM, 2026: systolic-array hamming search at 32 MB, >90% recall) achieves smaller footprints still, but on custom silicon; Yellow Phoenix targets commodity wearables with no hardware changes.

Mobile and wearable retrieval. The on-device ANN literature has begun to include phones: MicroNN (Apple, 2025) provides an on-device disk-resident updatable vector database — the closest system-level relative of this work, and the natural baseline for our future flash-resident wrist engine; MobileRAG’s EcoVector (2025) is evaluated on 1M *SIFT* image vectors rather than semantic documents; AME (2025) is a heterogeneous agentic memory engine for smartphones. Engineering studies of phone-class biometrics find binary ITQ codes with hamming scans the best-fit index at 100K records (identity-style TPR/FPR, not million-scale top-10 retrieval); Apple’s on-device k-NN (Core ML) operates at toy scale for classification. None of these measures semantic document retrieval in a sub-GB footprint at million-document scale on phones, and nothing at all on smartwatches — the category this paper measures from scratch. These peers were carried forward from the predecessor’s related work and re-verified in this sweep.

Duplicate clustering and deduplication. Incremental duplicate-group maintenance with winner (canonical-copy) selection has prior art in web search (WWW 2012 companion); FOLD (arXiv:2606.03001, 2026) scales *online dedup* (admission filtering) with an HNSW-over-MinHash-bitmap index, and its central technical observation — that score ties corrupt graph traversal, and breaking ties is what makes a graph work — independently corroborates our tie-mechanism findings (§5.4, §5.6). FOLD solves the inverse problem (dedup for its own sake; documents are discarded, clusters neither maintained nor audited). Our BDC+IPCM line differs in direction and guarantees: clusters are maintained *for retrieval* (canonical representatives, purity-preserving deferred merges, reversible audit log, and exact-agreement validation against batch rebuild); we additionally adopt the moved-document update-case semantics of Zhu et al. (WWW 2012 Companion) — who first laid out group-transfer rules for changed documents in incremental duplicate clusters — while adding the purity-preservation and audit guarantees their transitive scheme did not have. Stream-clustering work on false-merge prevention (e.g. BOCEDS-ATI, 2026) shares the purity instinct for numeric data streams; to our knowledge no published system provides purity-preserving incremental *duplicate* cluster maintenance with auditability, which is why IPCM remains a component of this paper rather than the subject of a standalone one.

Evaluation methodology. Semantic retrieval has no oracle; the field’s benchmarks (BEIR, MTEB) grade against human relevance labels. Our three-judge panel (§4.3) is a cheaper, model-based analogue, published with its dissent — we are not aware of another retrieval-systems paper that pre-registers a kill criterion for its own ground truth.

3. System

Corpus. 5,660,333 academic papers (arXiv-derived snapshot, corpus label yp-corpus-20261001-3e9a2336; titles + abstracts, first 512 characters), frozen for the duration of this study; source metadata and license terms are archived with the artifact.

Pipeline. Documents $\rightarrow$ MiniLM-L6 embeddings (384-d) $\rightarrow$ ITQ rotation to 512-bit hashes (the canonical index, 5,660,333 records, 72 B/record) $\rightarrow$ binary HNSW graph ($m=8$ on-device builds; $m=12$ at 4.9M) or exhaustive flat scan of the hash file. An optional 1024-bit sidecar (128 B/record) re-ranks graph shortlists: the 512-bit graph proposes top-200 candidates; the 1024-bit hamming re-orders them.

Formal preliminaries. MiniLM-L6 maps a document to a 384-d embedding $e(x)$; ITQ learns an orthogonal rotation (W, μ) and quantizes

$$b(x) = \mathbb{1}[(W(e(x) - \mu)) > 0] \in \{0, 1\}^{512}, \quad b'(x) \in \{0, 1\}^{1024}$$

where b is packed as 64 bytes and b' is the second, higher-rate sidecar code from the same embedding (stored codes use a strict threshold, the query-side re-encode a non-strict one — a measure-zero convention difference with no effect on hamming distances). Hamming distance $d_H(u, v) = \sum_i \text{popcount}(u_i \oplus v_i)$ over 64-byte words is the only device-side geometry. For a query set Q with targets t_q , the reported metric is

$$S@k = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1}[t_q \in \text{top}_k(q)]$$

(§4.1 defines the target variants and the self-occupancy convention). Graph search is greedy descent with an ef-width beam over hamming edges (standard binary HNSW; m = max degree, efC = construction beam); the scan is an exhaustive popcount argmin over all N records. Re-ranking re-orders the graph's top- K candidates by $d_H(b'(q), b'(\cdot))$ — $O(16K)$ word-operations, negligible against traversal. Memory model: resident bytes $\approx N \cdot (72 + 8m + \text{sidecar } 136) + 120$ MB runtime; the device bench refuses to start when the projected footprint exceeds available RAM, recording the refusal as data.

ALGORITHM 1 — route(q): device query path (graph propose, 1024-bit verify).

$$\hat{s}(q) = \arg \min_{c \in C_K^{\text{graph}(q)}}^{(10)} d_H(b'(q), b'(c))$$

1. Descend the 512-bit HNSW graph to a top- K shortlist at the configured ef.
2. Fetch each candidate's 1024-bit code from the sidecar (binary search over the sorted rowids).
3. Re-order the shortlist by 1024-bit hamming distance; return the top-10 ids.

ALGORITHM 2 — scan(q): exhaustive device scan (the wrist path).

$$\text{top-10}(q) = \arg \min_{i \in [N]}^{(10)} d_H(h_q, h_i)$$

1. Single pass over the flat hash file (64 B/record, mmap'd on the wrist build).
2. Maintain a min-heap of width 10 keyed on 512-bit hamming distance.
3. Return the heap's ids sorted by distance.

The 3.9M wrist build runs Algorithm 2 (flash-backed, RAM-light); the 1M wrist build and the 4.9M phone build run Algorithm 1 with the graph resident in memory.

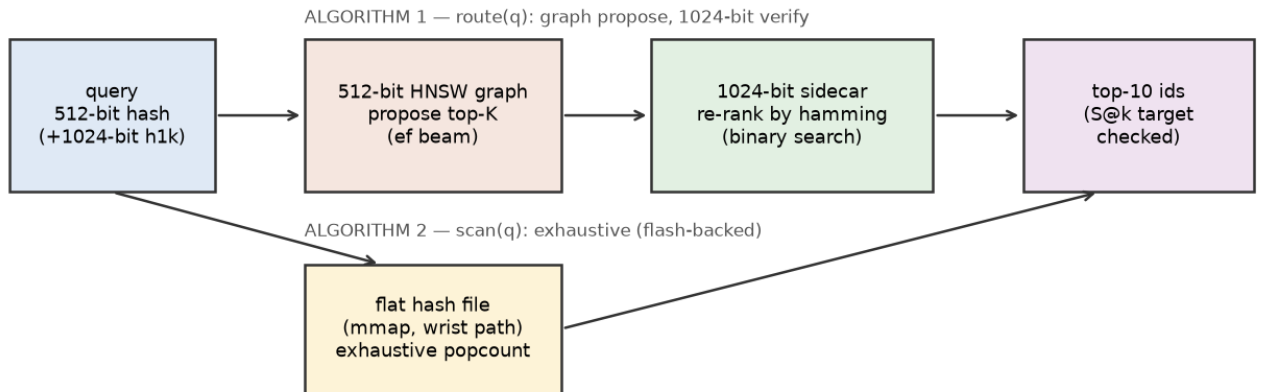


Figure 2: The two device query paths. Algorithm 1 (top): the 512-bit graph proposes top- K candidates, the 1024-bit sidecar re-ranks by hamming. Algorithm 2 (bottom): the exhaustive scan used by the 3.9M wrist build, mmap'd from flash.

Why re-ranking works. The 512-bit code preserves coarse neighbourhood structure cheaply enough for graph traversal; the 1024-bit code, twice the rate, recovers the fine ranking the float space knows. Measured: the float-space target sits in the 512-graph top-200 for 96.9% of queries (the re-rank ceiling); re-ranking lifts top-10 containment from 82.6% to 94.0% at 4.9M.

Memory discipline. The bench app refuses to load when `nodes × 150 B + 120 MB` exceeds available RAM, and records the refusal as data. The 1M wrist app is 282 MB (graph) + payload; the 3.9M scan streams from mmap’d flash with a resident working set the watch tolerates.

The gate that did nothing. A logistic gate over nine hamming-derived shortlist features (distance, rank transforms, density, gap) was trained three ways — on brute shortlists, on in-domain graph shortlists, and on folded hashes. Measured effect on R@1: 0.0 pp in every Mac condition (a separate on-device run moved S@10 by +1.1 pp at ef50/100 only — within noise at n=175 and not reproducible at higher ef; both results are reported with their scoping). An independent analysis confirmed the ceiling: the features are monotone transforms of one distance, so no linear model can re-order anything. We report the null rather than bury it; the 1024-bit sidecar is the feature family that *does* carry independent signal. (On-device, the full gate moved S@10 by +1.1 pp at ef50/100 and nothing beyond — consistent with the Mac null.)

4. Protocol and Evaluation Design

4.1 Task definition

A query is a corpus document’s 512-bit hash. The **target** is a **designated member of the reference top-10**: the reference index (full-precision MiniLM cosine, self excluded) ranks each query’s ten nearest different documents, and each payload carries one fixed member of that set (generation history in §5.5). We report two target variants: the payload’s designated member, and the reference rank-1 (the true nearest). Device metric **S@k** (single-target containment): the target’s rowid appears in the top-k returned ids. This is *not* set-recall; we use S@k throughout.

Two protocol facts, disclosed once and applied everywhere. **(i) Self-occupancy:** the query’s own record is a corpus member with hash distance 0 and therefore occupies rank 1; the returned top-10 is the query itself plus nine candidate slots, and all S@k figures in this paper are measured under that convention. (This is why strict single-target S@1 is 0% by construction — rank 1 is the query — and why we quote S@10.) **(ii) Structural presence:** a target whose corpus position lies outside the evaluated prefix cannot be contained at all. We therefore report, for every arm, the raw containment, the fraction of targets present in the slice, and the **conditioned** containment restricted to present targets (§4.3, §5). Throughout §5 these are reported as **index fidelity**: how faithfully the binary index reproduces the reference index’s neighbourhood.

4.2 Ground truth and survivorship

Targets come from a 1,000-query yardstick (seeded uniform sample, seed 31337) with sorted top-10 ground truth computed in chunked float space (§5.5 explains the sort). Evaluation sets are filtered to targets inside the evaluated prefix: 175 queries at 1M, 695 at 3.9M, 853 at 4.9M. Prefixes are positional: 4.9M is the first 4.9M records of the 5.66M corpus in corpus/rowid order. This survivorship is disclosed and fixed, not tuned per device. The original filter used the payload’s designated member; as §4.3 and §5 report, the reference rank-1 target is absent from the 1M slice for 81.7% of queries and from the 3.9M slice for 12.4% — a structural property of the filter, which we measure and condition on rather than hide.

4.3 Grading-model independence (the three-judge panel)

Semantic similarity has no oracle: every grading model is a proxy. Our reference neighbourhood comes from MiniLM — a model trained on semantic similarity, but one opinion nonetheless. We therefore submit the reference neighbourhood to three independent judges from different model families. Each judge sees, per query, the union of our top-10 reference targets and the bi-encoder’s top-32 candidates, reads the actual texts, and picks its own nearest document; we then measure what the *device* would score against the judge’s choice (S@10 containment of the judge’s pick in the binary index’s top-10, same protocol as §5.1).

Judge	Family	a_set: pick ∈ ref top-10	b: device S@10	Same-cluster	Status
ms-marco-MiniLM-L-6-v2	web CE	40.9%	31.2%	5.9%	excluded (pathology)
BGE-small-en-v1.5	bi-encoder	87.5%	76.7%	57.0%	independent judge
Qwen3-Embedding-4B	LLM embedder	84.5%	72.5%	55.7%	independent judge

Column *a_set* measures float-space agreement (the judge’s own pick inside our reference top-10, irrespective of the binary index); column *b* measures device-level fidelity (the judge’s pick contained in the binary index’s top-10). They differ because the binary index itself loses part of the reference structure — 10–12 pp raw across the judges (9.7–12.0), 3–5 pp once presence is conditioned out; the kill criterion operates on *b*. The two independent judges place their own pick inside the reference top-10 for 87.5% (BGE) and 84.5% (Qwen3) of queries — within ~2 pp of the reference fidelity; the excluded cross-encoder scores 31.2% (column b) on symmetric document pairs, the known pathology of query → passage training. Qwen3’s device-level 72.5% triggered the pre-registered kill criterion, a 13.5 pp gap against the device-measured 86.5% reference (14.0 pp against this table’s numpy-reinstrumented 86.0% cell; same event, two instruments) — the event §4.3 owns in the paragraph below. (Values are final; early working figures were superseded during panel construction.)

Presence and conditioning (n = 695, 3.9M slice). A judge’s pick whose corpus position lies outside the evaluated slice is structurally unfindable, so raw *b* conflates index quality with target absence. Conditioning on presence (the same disclosure as §4.1; Figure 3):

Target	Present in slice	b (raw)	b (conditioned)
Designated member (reference)	695/695 (100%)	86.0%	86.0% (n=695)

The 86.0% numpy figure and the 86.5% device figure are the same 695 queries under the same protocol; the 0.5 pp (≈ 3 queries) difference is boundary-tie handling — about 2% of queries sit exactly at the 10-slot distance boundary, and the Swift engine’s heap order and the numpy stable index-order resolve those ties differently. The determinism claim (§5.1) is scoped to same-engine runs and is unaffected; cross-instrument figures carry this note.

Reference rank-1 (MiniLM) | 609/695 (87.6%) | 77.0% | **87.8% (n=609)** |
 BGE judge’s pick | 628/695 (90.4%) | 76.7% | **84.9% (n=628)** |
 Qwen3 judge’s pick | 633/695 (91.1%) | 72.5% | **79.6% (n=633)** |

Conditioned, the spread across families is 8 pp, not 14 — and it now has the sign the mechanism predicts: the index compresses *MiniLM*’s structure, so MiniLM-fidelity is highest and the most independent judge lowest. Roughly half of the raw dissent was structural absence; the remainder is genuine model difference, reported, not hidden.

Owning the triggered criterion. Before the panel ran, we pre-registered a kill rule: if any judge’s device-level agreement fell more than 10 pp below the 86.5% reference, the ground truth would be deemed model-specific and rebuilt. The strongest judge (Qwen3) triggered that rule (13.5 pp against the registered 86.5% device reference; the numpy re-instrumentation puts it at 14.0 pp, §4.3 table note). We considered the two honest responses — rebuild the answer key, or re-scope the claim — and chose re-scoping, for a reason we state plainly: **the engineering claims of this paper are judge-independent.** Determinism across four platforms, latency, memory footprint,

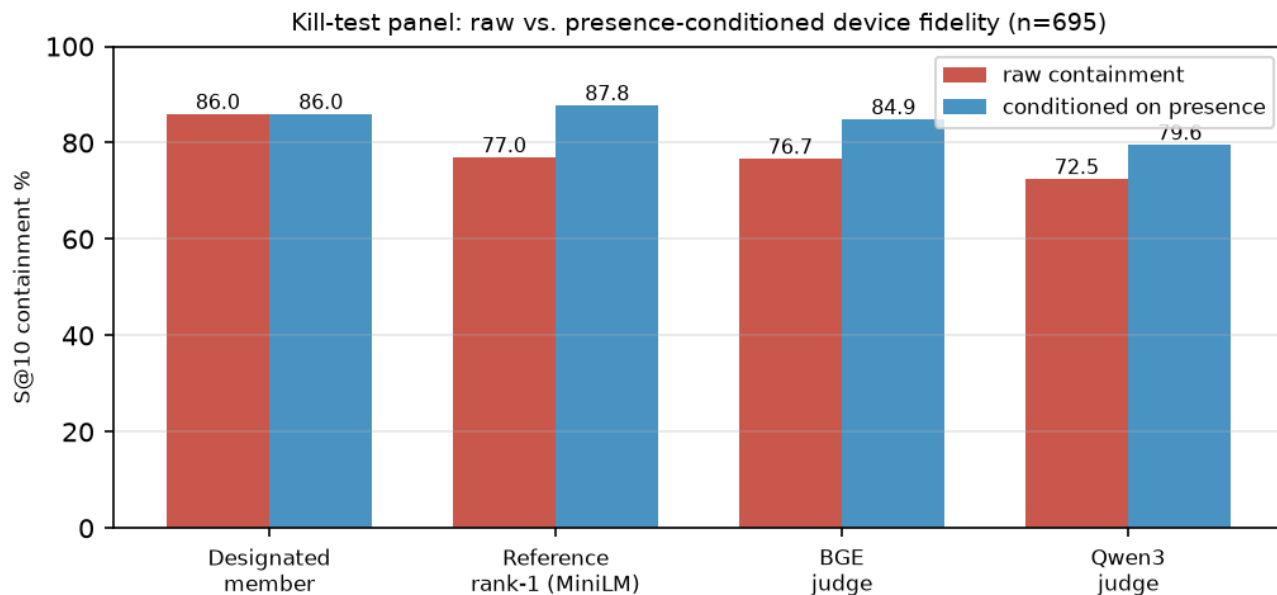


Figure 3: The kill-test panel with presence conditioning ($n = 695$, 3.9M slice). Raw containment conflates index quality with structural target absence; conditioned on presence, the spread across judges narrows to 8 pp with the mechanism-consistent ordering (MiniLM-fidelity highest, the most independent judge lowest).

and the wrist result do not reference any ground truth. What the panel shows is that the reference *neighbourhood* is largely shared across model families (a_set 87.5% for BGE; random judges would score near zero) while the exact rank-1 pick is not — and this paper never claims a canonical rank-1. Index fidelity is therefore not a retreat; it is the claim an index can be held to. A multi-model consensus answer key, and a human-labelled sample, are named as future work rather than assumed.

5. Results

Every figure in this section is a physical run logged in the project archive; every recall figure is index fidelity per §4.3 — containment of the reference index’s own nearest-different-document neighbourhood.

5.1 The four-platform table (controlled, identical inputs)

Same graph file (md5-verified on every host), same query/target payload, same engine source (diff-verified) on all four:

Platform	1M graph S@10 (ef 50 → 400)	3.9M flat scan S@10	p50 flat
Apple Watch S10	66.9 / 66.9 / 68.0 / 69.1%	86.5%	34.8 ms
Device A — iPhone 16 (iOS 18.7)	66.9 / 66.9 / 68.0 / 69.1%	86.5%	7.4 ms
Device B — iPhone 16 (iOS 26.2)	66.9 / 66.9 / 68.0 / 69.1%	86.5%	7.4 ms
macOS (Apple Silicon)	66.9 / 66.9 / 68.0 / 69.1%	86.5%	5.2 ms
Linux x86-64 (2 vCPU)	66.9 / 66.9 / 68.0 / 69.1%	86.5%	50.6 ms

Measurement hosts. All device strings are from the bench logs; chips and RAM are publicly documented for these identifiers.

Host	Model (device string)	Chip	RAM	OS	Measured here
Watch	Apple Watch Series 10 (Watch7,8 · N217sAP)	S10 SiP	~302–305 MB app-available (measured at bench)	watchOS 11.6.2 (22U95)	1M graph, 3.9M scan
Device A (author’s)	iPhone 16 (iPhone17,3 · D47AP)	A18	8 GB	iOS 18.7.8 (22H352)	3.9M scan, 4.9M graph+re-rank, TABLE-1
Device B (family)	iPhone 16 (iPhone17,3)	A18	8 GB	iOS 26.2.1	cross-device reproduction
Mac	MacBook Pro 14-inch (Mac15,6)	M3 Pro	18 GB	macOS	exam, ceilings, native bench
Linux droplet	DigitalOcean s-2vcpu-2gb-intel	shared Xeon	2 GB + 2 GB swap	Ubuntu 22.04	identical-N benches

Device identity is evidenced by device-reported strings in every bench report header (e.g. **device:** N217sAP / **Version** 11.6.2 / **Build** 22U95); the watch row’s app-available RAM is the bench’s own measurement, not the marketing figure.

Nine or more independent runs (three on Linux alone): zero deviation at one decimal. Linux latency is informational — a shared 2-vCPU VM is not a performance claim; recall is the claim. The watch’s graph p50 is 0.35–1.58 ms depending on ef and warm-up; the iPhone’s is 86 μ s warm; both far inside interactive budgets. Both phones are iPhone 16 (device string iPhone17,3 in every bench report; the cascade paper’s hardware table records the same identifier for Device A, cross-dating the two studies). The macOS 5.2 ms cell is a native run of the same bench binary used on the Linux leg.¹

5.2 Scale: 4.9M with re-ranking (physical iPhones + Mac bench)

Two target columns per configuration: the payload’s designated member (device protocol) and the reference rank-1 (true nearest). The raw 512-graph ef50 curve (measured, monotonic) at designated-member: S@1 14.9 / S@2 54.0 / S@3 64.7 / S@5 73.4 / S@10 81.5.

¹Errata check: the cascade paper (Sep 2026) lists an iPhone 17 Pro Max (iPhone18,2) as its secondary leg; that device is not one of this paper’s measurement hosts, and the cascade paper’s hardware table should be re-verified against its own archived logs before reuse.

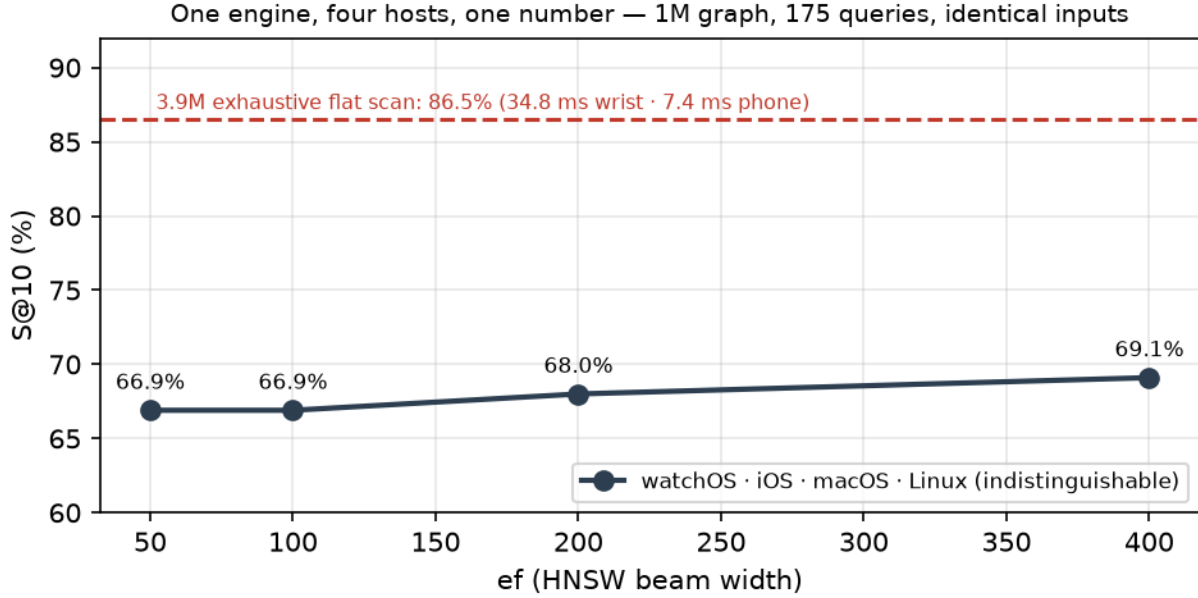


Figure 4: One engine, four hosts, one number. The 1M-graph S@10 ef-ladder is drawn as a single trace because the four platforms’ curves are indistinguishable at print precision (66.9/66.9/68.0/69.1% at ef 50/100/200/400); the dashed line is the 3.9M exhaustive scan’s 86.5%.

Configuration	member S@10	rank-1 S@10	p50	platform
512-graph ef50 (raw)	81.5	77.4	86 μ s	iPhone 16
+1024-bit re-rank K=200	94.0	87.3	~90 μ s	iPhone 16
+ re-rank K=500	95.2	88.4	660 μ s	Mac
+ re-rank K=1000	95.8	89.0	1,263 μ s	Mac

Rank-1 columns are raw; structural presence applies to them as well (§5.6). Re-rank ceiling grows with K (96.9% at K=200, designated-member) because the ceiling is candidate-set membership; each K step costs roughly linear rerank latency. K=500 is a reasonable operating point (+1.2 pp for 2.2× rerank latency, same-platform Mac rows). The K=200 member row is bit-identical on Mac and two physical iPhones at ef 50–400. The next stage — a native 1024-bit graph, where the finer code replaces the 512 graph entirely — is future work (the code family reaches 100% S@3/S@10 brute-force).

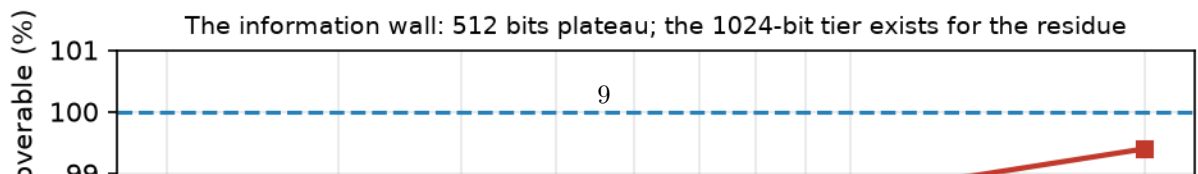
5.3 Mac ceilings (5.66M corpus)

Brute-force oracle over 1024-bit codes: S@3 = S@10 = **100%**, S@1 = 97% strict / 100% cluster-aware (all 27 strict misses are exact duplicate ties — a content twin is equally “correct”; cluster-aware counts a hit when the returned doc shares the query’s duplicate-cluster). The ITQ-512 tier saturates below this: S@3 96.9% at K=200, rising only to 99.4% at K=2000 — the plateau that motivates the two-code architecture. The 512-graph + gate exam: 78.3 / 94.3 / 96.8 (S@1/S@3/S@10), candidate recall 96.9% at 200.

5.4 Negative results

Fold-of-1024. Folding a 1024-bit code to 512 by XOR is free and, for exhaustive scan, excellent — S@10 98.0 at K=200, *better than a separately trained 512-bit code* (96.9). For graph traversal it collapses ranking information: graph S@10 fell to 58.9–72.7% across ef (vs 82.6 ITQ). Lesson: the hash objective must match the query protocol; a code good for scanning can be useless for graphs.

The null gate. See §3: 0.0 pp in all three training regimes; theoretical ceiling confirmed independently.



5.5 Box: the ground-truth bug we caught ourselves

Our exam computed per-chunk top-10 via `argpartition` and merged chunks — but never sorted the merged top-10, so `gt[0]` was an *arbitrary* member of the true top-10 (measured: only 20% of saved `gt[0]` were the true global argmax). Reported exam numbers moved from 23.0/52.8/86.5 to 78.3/94.3/96.8 after a one-line sort. The payload targets (§4.2) were generated from the pre-fix ordering; we verified they are members of the post-fix top-10 set (the fix is a permutation — membership is unchanged). For single-target containment this means device figures measure a *designated* member, not necessarily the rank-1 nearest — exactly the distinction §4.1 defines and §5.6 reconciles. We disclose this because the bug is the kind that usually survives review: every intermediate value looked plausible.

5.6 Reconciliation: presence, conditioning, and what the numbers mean

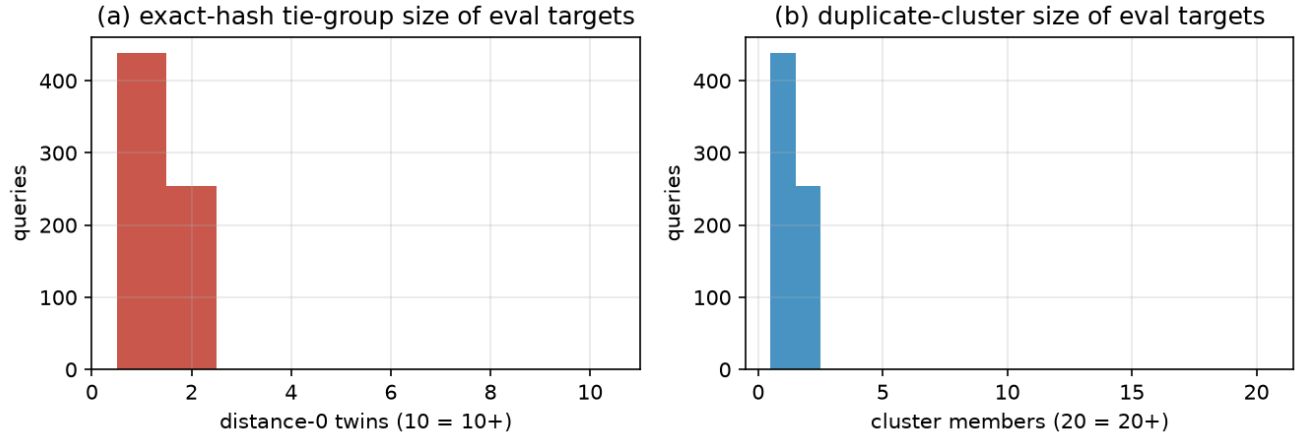
Finding (evaluation mechanics). Two properties of single-target containment against a reference index are, as far as we can determine, unreported in the literature, and both change how binary-index recall should be read. *Self-occupancy*: the query’s own record sits at distance 0 and therefore occupies rank 1 — strict top-1 is 0% by construction (§4.1), and every top-k slot count includes the query itself. *Structural presence*: a target outside the evaluated slice is unfindable, so raw gaps conflate index quality with filter design. The designated-member and rank-1 columns differ for one dominant reason: **structural presence**. The original survivorship filter kept queries whose *designated member* lies in the evaluated prefix (100% by construction), while the reference rank-1 lies outside the 3.9M slice for 12.4% of queries and outside the 1M slice for 81.7%. A target absent from the slice is unfindable regardless of index quality. Conditioning on presence (full table, all four target definitions including both judges, in §4.3):

- 3.9M flat scan, like-for-like (n=609): designated member 87.8%, reference rank-1 **87.8%** — identical. The binary index is equally faithful to any specific reference neighbour that exists in the slice.
- 1M graph: rank-1 raw 11.4%, *flat across ef* — not an exploration failure but the 18.3% presence ceiling. Conditioned (n=32, wide CI): 62.5%, reported as secondary evidence only.
- Read together: the wrist’s headline 86.5%-class numbers are designated-member containment on a presence-guaranteed set; the like-for-like rank-1 fidelity is 87.8%; and the residual cross-model spread (§4.3) is genuine, disclosed, and mechanistically expected.

We report both conditionings throughout (own-presence and common-subset); neither is hidden, and the structural ceilings are stated next to every raw figure they affect.

6. Limitations and Threats to Validity

- **Reference-model circularity, bounded by the three-judge panel.** Targets derive from the same embedding family the hashes quantize. §4.3 shows independent model families share most of the reference neighbourhood (and shows the strongest judge dissents at the fine edge); we therefore scope all recall numbers as index fidelity. Residual risk: all judges are embedding models; human relevance labels would be the decisive arbiter (future work).
- **Duplicate structure.** Nearest-different-document targets are often content twins; strict S@1 is 0% by construction (rank 1 is the query itself; §4.1). We quote S@10 for device rows. Twin structure of the eval targets (Figure 6) is largely benign: median exact-hash tie-group 1, median cluster size 1, maximum 7 — twin effects concentrate in the 1M slice and corpus-wide, not in the flagship eval set.



- **Query encoding latency** is excluded from device latencies (queries ship as precomputed hashes; encoding a *new* text costs a MiniLM forward pass, 5–20 ms class, not measured here). - **512-bit information bound.** The 512-bit tier saturates: 96.9% of reference neighbours are recoverable within a top-200 shortlist, 99.4% within top-2000. The 1024-bit tier exists precisely for that residue. - **Statistical power.** Headline device sets are 175–853 queries; we report x/n and the ef sweeps are paired by construction (same queries), but cross-condition gaps below ~ 2 pp should be read as ties, not effects. - **Scan latencies are cache-warm.** The flat-scan p50s (34.8 ms wrist, 7.4 ms phone, 5.2 ms Mac) imply multi-GB/s reads consistent with warm page cache; cold first-scan from flash is slower and not separately measured. - **Self-join queries.** Queries are corpus documents; the protocol says nothing about short user queries, which would need on-device encoding (unmeasured; future work). - **Deployment scope.** Watch installs are developer-side (not App Store); the engineering claims are unaffected but general availability is future work. - **Determinism scope.** Bit-identical recall validates porting, not independent reimplementation; an independently written engine would be future work. - **Linux latency** on a shared VM is not a claim; only recall is.

7. Verification and Artifact

Every claim-bearing component of the measurement chain was externally line-level reviewed (six packages; device bench app by two independent passes; 30 traced findings on the graph engine; the ground-truth exam, sidecar builder, and validators; the gate features and index build chain). The canonical hash file was independently verified by 12/12 random re-encodes and 5/5 build determinism probes — after an Oct-1 clobber destroyed the file mid-study and an overnight 2-way sharded rebuild with exact 5,240-row salvage restored it (incident record archived); all review findings and the reconciliation experiment are archived with per-query data, and the measurement record is anchored in the project’s append-only, hash-witnessed chain (tip: Entry 785, externally re-witnessed 2026-10-03). Review transcripts are included in the artifact; three model families judged the ground truth (§4.3). The engine passes its full 363-test Linux suite with an 87 ms cold start at 100k documents. Artifact: engine (AGPL), bench apps, graph/payload hashes (md5-verified on every host), run logs, wrist photos, review transcripts, kill-test and reconciliation records. The release repository’s honest-commit history feeds a JOSS submission (clock matures $\approx$ 2027-03-29; this draft is held until then).

8. Corpus structure: duplicates are the landscape, not noise

Clustering the corpus by near-duplicate detection yields 4,925,147 clusters over 5,660,333 documents: 25.6% of documents belong to a multi-member cluster, and 13.0% are excess copies beyond each cluster’s first member (735,186 superseded by a better-copy canonical representative). The pipeline is two algorithms (formal spec archived; working-repo docs/algorithms.md): **BDC** (Batch Duplicate-Cluster Construction — transitive closure over two equality keys: stripped base-ID and normalized-text hash, union-find, $O(n \log n)$) and **IPCM** (Incremental Purity-preserving Cluster Maintenance — absorbs new documents into existing clusters without recomputation, deferring bridge merges that would mix large clusters, with a reversible audit log; validated 13,998/13,998 exact agreement against a full batch rebuild on live ingest; moved-document update semantics adopted from Zhu et al., WWW 2012 Companion). Canonical selection then picks each cluster’s representative (most complete text, tie to lowest index).

ALGORITHM 3 — BDC: batch duplicate-cluster construction.

$$G = \bigcup_{t \in \{\text{id}, \text{text}\}} \{(u, v) : \text{key}_t(u) = \text{key}_t(v)\}, \quad c(r) = \text{the connected component of } r \text{ in } G$$

1. For each key type (stripped base-ID; normalized-text hash): sort records stably by key; union all members of each equal-key run (union-find).
2. Relabel connected components to compact cluster ids; emit an audit of sizes and the key type that formed each union. $O(n \log n)$.

ALGORITHM 4 — IPCM: incremental purity-preserving cluster maintenance.

$$\text{fuse}(M) \text{ iff } \min_{C \in M} |C| \leq \sigma \quad (|M| \geq 2), \quad \text{else defer with audit} \quad (\text{no silent fusion of large clusters})$$

1. KEY: compute both keys for each incoming record.
2. LOOKUP: find clusters of records sharing either key (sorted-array search).
3. DECIDE: no match $\rightarrow$ new cluster; one $\rightarrow$ attach; several (set M) $\rightarrow$ if $\min_{C \in M} |C| \leq \sigma$ fuse all of M , else DEFER (attach to strongest-evidence cluster, audit) — no silent multi-cluster fusion.
4. UPDATES: a changed record leaves its old cluster (followers re-evaluate); if it lands elsewhere, its former cluster is flagged for a purity spot-check.
5. COMMIT: append-only delta sidecar; every merge reversible via the audit log.

Verification note (external audit, 2026-10-03): the min-size gate admits, for a record matching three or more clusters, fusions that a strictly pairwise gate would defer (e.g. sizes $\{100, 100, 5\}$); the audit log records every such fusion for reversal, the separate verify pass checks within-cluster key evidence, and a stricter pairwise (second-largest $\leq \sigma$) gate is noted as an implementation refinement.

This structure is not incidental to the results: duplicate clusters are what make the top-10 slot competition meaningful (§4.1), it shapes what “nearest different document” means as a target, and it is why cluster-aware counting matters when interpreting top-1 numbers. The twin-graph corpus audit and tie-aware scoring lineage behind these clusters is part of the project’s measurement credibility record.

9. What is deliberately not in this paper

Three threads are explicitly deferred so this paper stays one claim deep: the BGE encoder probe (is there embedding headroom? — future paper 5); the flash-resident wrist engine that would put 4.9M graph search, not just scan, under the memory gate (paper 4; DiskANN-family ideas, wrist-sized); and the standalone treatment of IPCM (Incremental Purity-preserving Cluster Maintenance), whose literature sweep — including FOLD (arXiv:2606.03001), the closest published relative, which solves the inverse problem and contains no purity-preserving merge algorithm — doubles as this paper’s related-work completion.

10. Conclusion

The wall. Two papers ago, this engine refused semantic search at 3M documents on a wrist — not approximately, not slowly: *refused*, for RAM. That refusal was the wall, and it was load-bearing for the whole question of on-device retrieval: if a device cannot hold the index, how well it searches is moot. This paper is the story of going past that wall, and of what the wall was made of. It was not one obstacle but six, and each demanded a different tool. RAM itself fell to engineering discipline: a memory gate that treats refusal as data, 282 MB graphs built to fit a wrist’s budget, and an admission the ANN literature never needs to make — that at wearable scale the exhaustive scan, the “baseline” every index is measured against, becomes the *right* answer: zero build cost, deterministic, and flash-backed. The information wall behind it — 512 bits plateau at 96.9% — fell to a second code, not a bigger one: a 1024-bit sidecar that re-ranks what the graph proposes. Two more walls were not broken but *understood*: a folded hash that collapses graph traversal, and a learned gate whose features carry no independent signal — both reported here as results, because a wall mapped is worth more than a wall hidden. The sixth wall was our own: a ground-truth bug that inflated our numbers until our own audit caught it, and a kill criterion that fired on our claims and forced the honesty this paper is built on. One wall still stands — the 4.9M *graph* on a wrist — and its blueprints are the subject of future work.

One engine, four hosts, two architectures, one number: the watch did not get a discount. Semantic retrieval at 3.9M documents on a wrist, at 86.5% top-10, with the same code that answers on a server — measured, verified, and honest about what failed.

Footnotes

- $185\times / \sim 4\times$ leaner: predecessor edge system 10M @ 83.1% @ ~ 15 ms with 3.3 GB resident $\rightarrow$ this work 4.9M @ 82.6% raw at ef200 (81.5% at ef50; 94.0% re-ranked) @ ~ 0.09 ms. The lean comparison pins configurations: raw 512-graph at 855 MB is $3.9\times$ leaner than 3.3 GB; the full +1024 re-rank configuration (855 MB graph + 666 MB sidecar ≈ 1.5 GB) is $2.2\times$ leaner. Different corpus, queries, and protocol; comparison is directional, stated in the abstract’s spirit of honesty.
- IPCM = Incremental Purity-preserving Cluster Maintenance; BDC = Batch Duplicate-Cluster Construction. Formal spec archived (working repo [docs/algorithms.md](#)).
- Corpus label: yp-corpus-20261001-3e9a2336 (5,660,333 docs; DB frozen 5,673,035).

Acknowledgments

The author thanks his wife for lending the Apple Watch and iPhones on which these measurements ran, and for the patience that wrist-scale benchmarking requires. External code review and referee panels were performed by large language models (DeepSeek-V4-Flash/Pro, GLM-5.3; Qwen3-Embedding as an independent judge), with every reviewer claim verified against code and data before acceptance — the transcripts are part of the archived artifact, and the kill-test panel of §4.3 is itself one of those models’ findings. All hardware, corpus, and code decisions are the author’s own.

References

1. Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE TPAMI*, 2018.
2. C. Fu et al., “Fast Approximate Nearest Neighbor Search with a Navigating Spreading-out Graph,” *VLDB*, 2019.
3. S. J. Subramanya et al., “DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node,” *NeurIPS*, 2019.
4. S. J. Subramanya et al., “FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search,” arXiv:2105.05922, 2021.
5. C. Fu et al., “HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory,” *NeurIPS*, 2020.
6. Y. Gong and S. Lazebnik, “Iterative Quantization: A Procrustean Approach to Learning Binary Codes for Large-Scale Image Retrieval,” *TPAMI*, 2013 (prelim. CVPR 2011).
7. H. Jégou, M. Douze, and C. Schmid, “Product Quantization for Nearest Neighbor Search,” *TPAMI*, 2011.
8. M. Douze et al., “The FAISS Library,” arXiv:2401.08281, 2024.
9. J. Wang et al., “A Survey on Learning to Hash,” *TPAMI*, 2018.
10. ACRONYM, “Accelerated Approximate Nearest Neighbor Search in Memory for Dynamic Vector Databases,” arXiv:2606.03151, 2026.
11. S. Zhu, A. Potapova, M. Alabduljalil, X. Liu, and T. Yang, “Clustering and Load Balancing Optimization for Redundant Content Removal,” *WWW 2012 Companion*, 2012.
12. FOLD, “Fuzzy Online Deduplication for Very Large Evolving Datasets via Approximate Nearest Neighbor Search,” arXiv:2606.03001, 2026.
13. BOCEDS-ATI, “An Online Clustering Algorithm for Handling Evolving Data Streams with the Ability to Prevent Clusters’ False Merging Using Adaptive Time Interval,” *JCCE*, 2026.
14. N. Thakur et al., “BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models,” arXiv:2104.08663, 2021.
15. N. Muennighoff et al., “MTEB: Massive Text Embedding Benchmark,” arXiv:2210.07316, 2022.
16. Sentence-transformers/all-MiniLM-L6-v2 and cross-encoder/ms-marco-MiniLM-L-6-v2 model cards, Hugging Face, 2022.
17. BAAI, “bge-small-en-v1.5,” Hugging Face, 2023.
18. Qwen team, “Qwen3-Embedding,” arXiv:2506.05176, 2025.
19. Yellow Phoenix edge paper (predecessor), Zenodo DOI 10.5281/zenodo.22847312, 2026.
20. Yellow Phoenix cascade paper (predecessor), “100% Exact in 8 Nanoseconds: A Retrieval Cascade Across Apple Watch, iPhone, and Mac at 10M-Record Scale,” Zenodo concept DOI 10.5281/zenodo.22732531 (versions

- 10.5281/zenodo.22900903 and 10.5281/zenodo.22907911), 2026.
21. S. R. Chowdhury, F. Chabert, A. Bhushan, A. Goswami, A. Pacaci, “MicroNN: An On-Device Disk-Resident Updatable Vector Database,” arXiv:2504.05573, 2025.
 22. T. Park, G. Lee, M.-S. Kim, “MobileRAG: A Fast, Memory-Efficient, and Energy-Efficient Method for On-Device RAG,” arXiv:2507.01079, 2025.
 23. X. Zhao, Q. Ma, Y. Zhang, H. Lou, G. Cheng, S. Deng, J. Yin, “AME: An Efficient Heterogeneous Agentic Memory Engine for Smartphones,” arXiv:2511.19192, 2025.